

Apostila Completa e Didática de Flutter para Iniciantes

Introdução

Bem-vindo(a) à sua jornada no mundo do **Flutter**!

Esta apostila foi cuidadosamente elaborada para ser o seu guia completo e didático, partindo do zero até a criação de aplicativos funcionais e robustos. O Flutter, o *framework* de UI do Google, permite que você construa belos aplicativos nativamente compilados para dispositivos móveis, web e desktop a partir de uma única base de código.

Nosso objetivo é desmistificar o desenvolvimento Flutter, apresentando cada conceito de forma clara, objetiva e com a profundidade necessária para que você se sinta confiante em seus primeiros projetos.

O que esperar desta apostila:

- **Linguagem Simples:** Explicações passo a passo, como um professor guiando o aluno.
- **Progressão Lógica:** Capítulos estruturados para construir seu conhecimento de forma sólida.
- **Prática Imediata:** Exercícios ao final de cada capítulo e exemplos práticos completos.

Prepare-se para mergulhar no Dart e nos Widgets. O futuro do desenvolvimento multiplataforma começa agora!

Índice

Capítulo	Título	Conteúdo Principal
1	Introdução ao Flutter	O que é Flutter, Dart e por que utilizá-lo.
2	Preparando o Ambiente	Instalação e configuração (Windows, Linux, macOS). Estrutura de um projeto Flutter.
3	Fundamentos de Widgets	Widgets fundamentais (<code>Text</code> , <code>Column</code> , <code>Row</code> , <code>Container</code> , <code>Image</code> , etc.).
4	Gerenciamento de Estado Básico	<code>StatelessWidget</code> vs <code>StatefulWidget</code> e o método <code>setState</code> .
5	Navegação entre Telas	Navegação básica (<code>Navigator 1.0</code>) e rotas nomeadas.
6	Gerenciamento de Estado Intermediário	Introdução a <code>Provider</code> e <code>Riverpod</code> .
7	Listas e Dados Dinâmicos	Uso de listas e <code>ListView.builder</code> .
8	Interação com o Usuário	Formulários, validações e <code>TextEditingController</code> .
9	Comunicação com o Mundo Exterior	Consumo de API REST com o pacote <code>http</code> .
10	Persistência de Dados Simples	Armazenamento local com <code>SharedPreferences</code> .
11	Boas Práticas e Arquitetura	Organização de pastas, nomenclatura, dicas de performance e debug.
12	Glossário de Termos	Definições dos termos mais usados em Flutter.
Apêndice A	Exemplo Prático: App Contador	Código completo e explicação do App Contador.
Apêndice B	Exemplo Prático: Lista de Tarefas	Código completo e explicação do App de Lista de Tarefas.

Capítulo	Título	Conteúdo Principal
Apêndice C	Exemplo Prático: Consumo de API	Código completo e explicação do App de Consumo de API.

Capítulo 1: Introdução ao Flutter

1.1. O que é Flutter?

O **Flutter** é um *framework* de desenvolvimento de interface de usuário (UI) de código aberto, criado pelo Google. Ele permite que você crie aplicativos nativamente compilados para **múltiplas plataformas** a partir de uma única base de código.

Em termos simples, o Flutter é como um kit de ferramentas que permite desenhar e construir a aparência e o comportamento do seu aplicativo de forma rápida e eficiente.

1.1.1. A Linguagem de Programação: Dart

O Flutter utiliza a linguagem de programação **Dart**, também desenvolvida pelo Google. O Dart é uma linguagem otimizada para o desenvolvimento de UI, conhecida por sua sintaxe limpa e por ser compilada para código nativo, o que garante um desempenho excelente.

Por que Dart?

- **Compilação Nativa:** O Dart pode ser compilado para código nativo (ARM e x86), o que significa que seus aplicativos rodam diretamente no hardware, sem a necessidade de uma ponte de comunicação lenta, como ocorre em outras tecnologias.
- **Hot Reload:** Uma das funcionalidades mais amadas do Flutter. O *Hot Reload* permite que você veja as alterações no código refletidas instantaneamente no aplicativo em execução, acelerando drasticamente o ciclo de desenvolvimento.

- **Otimizado para UI:** A linguagem possui recursos que facilitam a criação de interfaces reativas e fluidas.

1.2. Por que Utilizar o Flutter?

A popularidade do Flutter cresceu exponencialmente nos últimos anos devido a uma série de vantagens que ele oferece aos desenvolvedores e às empresas.

1.2.1. Desenvolvimento Multiplataforma (Cross-Platform)

Esta é a principal razão para a adoção do Flutter. Com uma única base de código em Dart, você pode gerar aplicativos para:

- **Mobile:** Android e iOS
- **Web:** Navegadores (Chrome, Firefox, etc.)
- **Desktop:** Windows, macOS e Linux
- **Embarcados:** Dispositivos como carros e eletrodomésticos

Isso significa menos tempo de desenvolvimento e manutenção, pois você não precisa de equipes separadas para cada plataforma.

1.2.2. Desempenho Nativo

Ao contrário de outras soluções *cross-platform* que dependem de *webviews* ou pontes de comunicação, o Flutter compila para código nativo. Ele utiliza seu próprio motor de renderização (o **Skia**, o mesmo usado no Chrome e no Android), o que lhe dá controle total sobre cada pixel na tela. O resultado são aplicativos com desempenho indistinguível de um aplicativo nativo.

1.2.3. Interface de Usuário Expressiva e Flexível

O Flutter é construído em torno do conceito de **Widgets**. Tudo no Flutter é um Widget. Isso permite que você crie interfaces complexas e altamente personalizadas com facilidade.

Característica	Descrição	Vantagem
Hot Reload	Recarrega o código instantaneamente no app em execução.	Desenvolvimento rápido e interativo.
Widgets Próprios	Não depende dos componentes nativos do sistema operacional.	Consistência visual em todas as plataformas.
Código Nativo	Compila para código de máquina.	Desempenho e fluidez de um app nativo.
Comunidade Forte	Grande e crescente comunidade de desenvolvedores.	Suporte, bibliotecas e recursos abundantes.

1.3. O Ecossistema Flutter

O Flutter não é apenas o *framework*; ele é um ecossistema completo que inclui:

- **Flutter SDK (Software Development Kit):** O conjunto de ferramentas que permite compilar seu código.
- **Dart SDK:** O kit de desenvolvimento da linguagem Dart.
- **Flutter Engine:** O motor de renderização de baixo nível (Skia).
- **Foundation Library:** A biblioteca principal que fornece classes e funções básicas.
- **Widgets:** O catálogo de componentes de UI (Material e Cupertino).

1.4. Exercícios do Capítulo 1

1. Em suas próprias palavras, defina o que é o Flutter e qual a sua principal vantagem.
2. Qual a linguagem de programação utilizada pelo Flutter e por que ela é considerada ideal para o desenvolvimento de UI?
3. Explique o conceito de *Hot Reload* e como ele beneficia o processo de desenvolvimento.
4. Cite pelo menos três plataformas para as quais o Flutter pode compilar um aplicativo a partir de uma única base de código.

5. Pesquise e anote o nome de uma grande empresa que utiliza Flutter em seus aplicativos.

Próximo Capítulo: No Capítulo 2, você aprenderá a instalar e configurar o ambiente de desenvolvimento do Flutter em seu computador, independentemente do seu sistema operacional.

Capítulo 2: Preparando o Ambiente

A jornada no desenvolvimento Flutter começa com a configuração correta do seu ambiente de trabalho. Este capítulo guiará você passo a passo na instalação do Flutter SDK em diferentes sistemas operacionais e apresentará a estrutura básica de um projeto.

2.1. Requisitos Mínimos

Antes de começar, certifique-se de que seu sistema atende aos requisitos básicos:

Sistema Operacional	Espaço em Disco	Ferramentas Necessárias
Windows	1.64 GB (sem IDE/ferramentas)	Windows PowerShell 5.0 ou mais recente, Git para Windows.
macOS	2.8 GB (sem IDE/ferramentas)	Xcode (para desenvolvimento iOS), Git.
Linux	600 MB (sem IDE/ferramentas)	Ferramentas de linha de comando (bash, curl, unzip, etc.), Git.

Em todos os casos, você precisará de um editor de código, sendo o **VS Code** ou o **Android Studio** as opções mais recomendadas.

2.2. Instalação do Flutter SDK

O processo de instalação envolve baixar o SDK e adicioná-lo à variável de ambiente `PATH` do seu sistema.

2.2.1. Instalação no Windows

1. **Baixe o SDK:** Acesse o site oficial do Flutter (<https://docs.flutter.dev/install/windows>) e baixe a versão estável mais recente do SDK.
2. **Extraia o Arquivo:** Descompacte o arquivo `flutter_windows_vX.X.X-stable.zip` em um local de fácil acesso, como `C:\src\flutter`. **Importante:** Não instale em um diretório como `C:\Program Files\`, pois pode exigir permissões elevadas.
3. **Atualize o PATH:**
 - Procure por “Editar as variáveis de ambiente do sistema” no menu Iniciar.
 - Clique em “Variáveis de Ambiente...”.
 - Na seção “Variáveis do usuário”, encontre a variável `Path` e clique em “Editar”.
 - Adicione o caminho para a pasta `bin` dentro da pasta `flutter` (ex: `C:\src\flutter\bin`).
4. **Verifique a Instalação:** Abra o **PowerShell** e execute o comando:

```
flutter
```

Se a instalação estiver correta, você verá a lista de comandos do Flutter.

2.2.2. Instalação no macOS

1. **Baixe o SDK:** Acesse o site oficial do Flutter (<https://docs.flutter.dev/install/macos>) e baixe a versão estável mais recente.
2. **Extraia o Arquivo:** Descompacte o arquivo em um local como `~/development/flutter`.

3. **Atualize o PATH:** Abra o terminal e adicione o caminho do Flutter ao seu `PATH`. Se você usa `zsh` (padrão no macOS moderno), edite o arquivo `~/.zshrc`:

```
export PATH="$PATH:~/development/flutter/bin"
```

Salve o arquivo e execute `source ~/.zshrc` para aplicar as mudanças.

4. **Verifique a Instalação:** No terminal, execute:

```
flutter
```

2.2.3. Instalação no Linux

1. **Baixe o SDK:** Acesse o site oficial do Flutter (<https://docs.flutter.dev/install/linux>) e baixe a versão estável.
2. **Extraia o Arquivo:** Descompacte o arquivo em um local como `~/development/flutter`.
3. **Atualize o PATH:** Edite o arquivo `~/.bashrc` ou `~/.zshrc` e adicione a linha:

```
export PATH="$PATH:~/development/flutter/bin"
```

Salve o arquivo e execute `source ~/.bashrc` (ou `source ~/.zshrc`).

4. **Instale Dependências:** Instale as dependências necessárias (geralmente `unzip`, `git`, `curl`, `libglu1-mesa` e `mesa-utils`).
5. **Verifique a Instalação:** No terminal, execute:

```
flutter
```


2.3. O Comando `flutter doctor`

Após a instalação do SDK, o comando mais importante é o `flutter doctor`. Ele verifica seu ambiente e informa quais ferramentas adicionais você precisa instalar ou configurar (como Android Studio, Xcode, ou plugins de IDE).

```
flutter doctor
```

O resultado ideal é uma lista de *checks* com um `[✓]` verde. Se houver algum `[!]`, siga as instruções fornecidas pelo comando para corrigir o problema.

2.4. Estrutura de um Projeto Flutter

Ao criar um novo projeto com `flutter create nome_do_projeto`, a seguinte estrutura de pastas é gerada:

Pasta/Arquivo	Descrição
android/	Contém o código específico para a plataforma Android (Kotlin/Java).
ios/	Contém o código específico para a plataforma iOS (Swift/Objective-C).
lib/	Onde você passará a maior parte do seu tempo. Contém o código-fonte Dart do seu aplicativo.
test/	Contém os testes automatizados para o seu código Dart.
web/	Contém os arquivos de <i>build</i> para a versão web do seu aplicativo.
pubspec.yaml	O arquivo de configuração do projeto. Define dependências, ativos (imagens, fontes) e metadados.
README.md	Um arquivo Markdown com informações sobre o projeto.
.gitignore	Define quais arquivos e pastas devem ser ignorados pelo Git.

2.4.1. O Arquivo `pubspec.yaml`

Este arquivo é crucial. Ele funciona como o “manifesto” do seu projeto.

```
name: meu_primeiro_app
description: Um novo projeto Flutter.
version: 1.0.0+1

environment:
  sdk: '>=3.0.0 <4.0.0' # Versão mínima e máxima do Dart SDK

dependencies:
  flutter:
    sdk: flutter
  # Adicione suas bibliotecas aqui, por exemplo:
  http: ^1.1.0
  provider: ^6.0.5

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^3.0.0 # Ferramenta de análise de código

flutter:
  uses-material-design: true
  # Para adicionar imagens e fontes:
  assets:
    - assets/images/
```

Sempre que você adicionar uma nova dependência, deve executar o comando `flutter pub get` no terminal para baixá-la.

2.4.2. A Pasta `lib` e o Arquivo `main.dart`

O ponto de entrada de todo aplicativo Flutter é a função `main()` dentro do arquivo `lib/main.dart`.

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp()); // Inicia a execução do aplicativo
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Meu Primeiro App Flutter',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: const MyHomePage(title: 'Página Inicial'),
    );
  }
}
// ... o restante do código do app
```

2.5. Exercícios do Capítulo 2

1. Após instalar o Flutter SDK, qual comando você deve executar para verificar se todas as dependências do ambiente estão corretas?
2. Qual é o propósito do arquivo `pubspec.yaml` em um projeto Flutter?
3. Se você deseja adicionar uma nova biblioteca (pacote) ao seu projeto, em qual seção do `pubspec.yaml` você a colocaria?
4. Qual é o nome da pasta que contém a maior parte do código-fonte Dart do seu aplicativo?
5. Explique a importância da função `main()` e do método `runApp()` no ciclo de vida de um aplicativo Flutter.

Próximo Capítulo: No Capítulo 3, mergulharemos no coração do Flutter: os Widgets. Você aprenderá a usar os componentes fundamentais para construir sua primeira interface.

Capítulo 3: Fundamentos de Widgets

Se o Dart é a linguagem do Flutter, os **Widgets** são os blocos de construção. No Flutter, **tudo é um Widget**. Seja um texto, um botão, um layout, uma animação ou até mesmo o aplicativo inteiro, tudo é representado por um Widget.

3.1. A Filosofia “Tudo é um Widget”

Um Widget é uma descrição de uma parte da interface do usuário. Eles são imutáveis (não mudam após serem criados) e são usados para descrever como a visualização deve se parecer, dada a sua configuração e estado atual.

Existem dois tipos principais de Widgets:

- Widgets Visíveis (Renderizados):** São aqueles que o usuário vê na tela, como `Text`, `Image`, `Button`.
- Widgets Invisíveis (Estruturais/Layout):** São aqueles que não aparecem diretamente, mas controlam como outros Widgets são dispostos, como `Column`, `Row`, `Padding`, `Container`.

3.2. O Widget Estrutural: Scaffold

O `Scaffold` é o Widget fundamental para a criação de uma tela. Ele fornece a estrutura básica para a maioria dos aplicativos Material Design.

Propriedade	Descrição
<code>appBar</code>	A barra de título na parte superior da tela.
<code>body</code>	O conteúdo principal da tela, onde a maior parte da sua UI estará.
<code>floatingActionButton</code>	Um botão de ação flutuante, geralmente usado para a ação primária da tela.
<code>drawer</code>	Um painel lateral que desliza para fora.

Exemplo de Uso do `Scaffold`:

```
import 'package:flutter/material.dart';

class MinhaTela extends StatelessWidget {
  const MinhaTela({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Minha Primeira Tela'),
        backgroundColor: Colors.blue,
      ),
      body: const Center(
        child: Text('Olá, Flutter!'),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          // Ação do botão
        },
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

3.3. Widgets de Layout: Column e Row

O Flutter usa um sistema de layout baseado em restrições. Os Widgets `Column` e `Row` são os mais importantes para organizar outros Widgets verticalmente e horizontalmente, respectivamente.

3.3.1. Column (Vertical)

O `Column` organiza seus Widgets filhos em uma única coluna vertical.

3.3.2. Row (Horizontal)

O `Row` organiza seus Widgets filhos em uma única linha horizontal.

Ambos os Widgets possuem duas propriedades principais para alinhamento:

- `mainAxisAlignment` : Controla como os filhos são dispostos ao longo do eixo principal (vertical para `Column`, horizontal para `Row`).
 - *Valores comuns:* `start`, `end`, `center`, `spaceBetween`, `spaceAround`, `spaceEvenly`.
- `crossAxisAlignment` : Controla como os filhos são dispostos ao longo do eixo cruzado (horizontal para `Column`, vertical para `Row`).
 - *Valores comuns:* `start`, `end`, `center`, `stretch`.

Exemplo de `Column` e `Row`:

```
// Exemplo de Column
Column(
  mainAxisAlignment: MainAxisAlignment.center, // Centraliza verticalmente
  crossAxisAlignment: CrossAxisAlignment.start, // Alinha à esquerda
  horizontalmente
  children: const <Widget>[
    Text('Primeiro Item'),
    Text('Segundo Item'),
    Text('Terceiro Item'),
  ],
)

// Exemplo de Row
Row(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly, // Espaço igual entre os
  itens
  children: const <Widget>[
    Icon(Icons.star),
    Icon(Icons.star),
    Icon(Icons.star),
  ],
)
```

3.4. O Widget “Coringa” : `Container`

O `Container` é um dos Widgets mais versáteis. Ele pode ser usado para:

- **Estilizar:** Adicionar cor, bordas, sombras (`decoration`).
- **Dimensionar:** Definir largura e altura (`width`, `height`).

- **Espaçamento:** Adicionar margem (`margin`) e preenchimento (`padding`).

Exemplo de `Container` :

```
Container(  
  width: 100,  
  height: 100,  
  margin: const EdgeInsets.all(10.0), // Margem externa  
  padding: const EdgeInsets.all(20.0), // Preenchimento interno  
  decoration: BoxDecoration(  
    color: Colors.red,  
    borderRadius: BorderRadius.circular(10), // Cantos arredondados  
    boxShadow: const [  
      BoxShadow(  
        color: Colors.black26,  
        blurRadius: 5.0,  
      ),  
    ],  
  ),  
  child: const Center(  
    child: Text('Caixa'),  
  ),  
)
```

3.5. Widgets de Conteúdo Essenciais

Widget	Descrição	Propriedades Chave
Text	Exibe uma string de texto.	<code>style</code> (para cor, tamanho, fonte), <code>textAlign</code> .
Image	Exibe uma imagem. Pode ser de um ativo local (<code>AssetImage</code>) ou da internet (<code>NetworkImage</code>).	<code>image</code> , <code>width</code> , <code>height</code> , <code>fit</code> (como a imagem deve preencher o espaço).
Icon	Exibe um ícone do conjunto de ícones do Material Design.	<code>icon</code> (o ícone a ser exibido), <code>color</code> , <code>size</code> .
Padding	Adiciona espaço (preenchimento) ao redor de um Widget.	<code>padding</code> (o tipo de espaçamento), <code>child</code> (o Widget a ser espaçado).

Exemplo de Image :

```
// Imagem da internet
Image.network(
  'https://picsum.photos/250?image=9',
  width: 150,
  height: 150,
  fit: BoxFit.cover,
)

// Imagem de um ativo local (requer configuração no pubspec.yaml)
Image.asset('assets/images/logo.png')
```

3.6. Exercícios do Capítulo 3

1. Qual é a principal função do Widget `Scaffold` ?
2. Qual a diferença fundamental entre `Column` e `Row` ?
3. Você precisa centralizar um texto na tela. Qual Widget de layout você usaria para envolver o `Text` ?
4. Explique a diferença entre as propriedades `mainAxisAlignment` e `crossAxisAlignment` em um `Column` .
5. Crie um código Dart que utilize um `Container` com fundo azul, margem de 15 pixels e um `Text` centralizado dentro dele com a palavra “Flutter” .

Próximo Capítulo: No Capítulo 4, você dará um passo crucial para entender como os Widgets interagem com os dados, aprendendo a diferença entre Widgets sem estado (`StatelessWidget`) e Widgets com estado (`StatefulWidget`).

Capítulo 4: Gerenciamento de Estado Básico

Até agora, aprendemos a construir interfaces estáticas. No entanto, a maioria dos aplicativos precisa de interatividade, ou seja, a interface deve mudar em resposta a

ações do usuário ou a eventos externos. É aqui que entra o conceito de **Estado**.

4.1. O Conceito de Estado

Em Flutter, o **Estado** de um Widget é o conjunto de dados que pode mudar durante a vida útil do Widget e que afeta a aparência da interface. Por exemplo, o texto em um campo de entrada, a cor de um botão ou o valor de um contador são todos exemplos de estado.

Para lidar com essa mudança, o Flutter divide seus Widgets em duas categorias principais:

4.2. StatelessWidget (Widget Sem Estado)

Um `StatelessWidget` é um Widget cuja aparência e comportamento são **imutáveis** após sua criação. Ele não armazena nenhum estado que possa mudar.

- **Características:**
 - Recebe informações (parâmetros) no seu construtor.
 - O método `build()` é chamado apenas uma vez.
 - Ideal para Widgets puramente de apresentação (como `Text`, `Icon`, `Container` com cor fixa).

Exemplo de `StatelessWidget` :

```
import 'package:flutter/material.dart';

class TituloEstatico extends StatelessWidget {
  final String titulo;

  // O construtor recebe o título
  const TituloEstatico({super.key, required this.titulo});

  @override
  Widget build(BuildContext context) {
    // O Widget sempre retornará o mesmo Text, independentemente de qualquer
    ação
    return Text(
      titulo,
      style: const TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
    );
  }
}
```

4.3. StatefulWidget (Widget Com Estado)

Um `StatefulWidget` é um Widget que pode mudar sua aparência em resposta a eventos. Ele armazena um estado que pode ser lido de forma síncrona quando o Widget é construído e pode ser alterado durante a vida útil do Widget.

Um `StatefulWidget` é composto por duas classes:

1. **O Widget em si:** A classe que herda de `StatefulWidget`. Ela é imutável e sua única função é criar a instância da classe `State`.
 2. **A Classe `state`:** A classe que herda de `State<SeuWidget>`. É aqui que o estado mutável (`_contador`, por exemplo) e o método `build()` residem.
- **Características:**
 - Possui um estado interno que pode ser alterado.
 - O método `build()` pode ser chamado múltiplas vezes.
 - Ideal para Widgets interativos (como botões, caixas de seleção, formulários).

4.4. O Ciclo de Vida do StatefulWidget e o setState

O segredo para a reatividade no Flutter está no método `setState()`.

Quando um dado que afeta a interface muda, você deve envolver a alteração dentro de uma chamada a `setState()`.

```
// Dentro da classe _ContadorPageState
void _incrementarContador() {
  // 1. Altere o estado (o dado)
  // 2. Chame setState() para notificar o Flutter
  setState(() {
    _contador++;
  });
}
```

Ao chamar `setState()`, o Flutter faz duas coisas:

1. Marca o Widget como “sujo” (dirty).
2. Agenda a reconstrução do Widget, chamando novamente o método `build()`.

Exemplo Prático: O App Contador

Este é o primeiro exemplo prático, que demonstra a essência do `StatefulWidget` e do `setState`.

```

import 'package:flutter/material.dart';

class ContadorPage extends StatefulWidget {
  const ContadorPage({super.key, required this.title});

  final String title;

  @override
  State<ContadorPage> createState() => _ContadorPageState();
}

class _ContadorPageState extends State<ContadorPage> {
  // O estado mutável do Widget
  int _contador = 0;

  // Função que altera o estado e notifica o Flutter
  void _incrementarContador() {
    setState(() {
      // Esta linha altera o estado
      _contador++;
    });
  }

  @override
  Widget build(BuildContext context) {
    // O build é chamado sempre que o estado muda (após o setState)
    return Scaffold(
      appBar: AppBar(
        title: Text(widget.title),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            const Text(
              'Você pressionou o botão este número de vezes:',
            ),
            Text(
              // O valor do contador é exibido aqui
              '$_contador',
              style: Theme.of(context).textTheme.headlineMedium,
            ),
          ],
        ),
      ),
      floatingActionButton: FloatingActionButton(

```

```

        onPressed: _incrementarContador, // Chama a função que altera o estado
        tooltip: 'Incrementar',
        child: const Icon(Icons.add),
      ),
    );
  }
}

```

4.5. Comparação: StatelessWidget vs StatefulWidget

Característica	StatelessWidget	StatefulWidget
Estado Interno	Não possui.	Possui (na classe <code>State</code>).
Mutabilidade	Imutável.	Mutável (o estado é mutável).
Reconstrução	O <code>build()</code> é chamado uma vez.	O <code>build()</code> é chamado sempre que <code>setState()</code> é invocado.
Uso Comum	Textos, ícones, imagens estáticas.	Botões, caixas de seleção, formulários, contadores.

4.6. Exercícios do Capítulo 4

1. Defina o que é o “Estado” de um Widget em Flutter.
2. Qual tipo de Widget você usaria para exibir um logotipo estático no cabeçalho do seu aplicativo? Justifique.
3. Qual tipo de Widget você usaria para criar um botão que muda de cor ao ser pressionado? Justifique.
4. Explique o papel do método `setState()` no `StatefulWidget`.
5. Modifique o App Contador para que ele também tenha um botão de decremento que subtraia 1 do contador.

Próximo Capítulo: No Capítulo 5, você aprenderá a organizar seu aplicativo em múltiplas telas e a navegar entre elas.

Capítulo 5: Navegação entre Telas

A maioria dos aplicativos é composta por mais de uma tela. A navegação é o processo de gerenciar a pilha de telas (ou *rotas*) que o usuário está visualizando. O Flutter utiliza o Widget `Navigator` para gerenciar essa pilha.

5.1. O Conceito de Rotas e Pilha

Pense na navegação como uma **pilha de cartas**. Cada carta é uma tela (ou *rota*).

- Quando você navega para uma nova tela, você **empilha** uma nova carta no topo.
- Quando você volta, você **desempilha** a carta do topo.

O `Navigator` é o Widget que gerencia essa pilha.

5.2. Navegação Básica (Navigator 1.0)

O `Navigator 1.0` é a forma mais tradicional e simples de navegação, ideal para a maioria dos aplicativos iniciantes. Ele usa métodos estáticos para empurrar (`push`) e remover (`pop`) rotas da pilha.

5.2.1. Navegando para uma Nova Tela (`push`)

Para ir para uma nova tela, usamos o método `Navigator.push()`. Este método requer um `BuildContext` (o contexto do Widget atual) e uma `Route` (a descrição da nova tela).

A forma mais comum de criar uma `Route` é usando o `MaterialPageRoute`.

```
// Exemplo de navegação para a SegundaTela
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => const SegundaTela(),
  ),
);
```

5.2.2. Voltando para a Tela Anterior (pop)

Para voltar para a tela anterior, usamos o método `Navigator.pop()`.

```
// Exemplo de retorno
Navigator.pop(context);
```

5.2.3. Passando Dados para a Próxima Tela

Para passar dados para a próxima tela, basta incluí-los no construtor da nova tela.

Primeira Tela:

```
// Passando o dado 'Olá da primeira tela'
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => const SegundaTela(mensagem: 'Olá da primeira tela'),
  ),
);
```

Segunda Tela:

```
class SegundaTela extends StatelessWidget {
  final String mensagem;

  const SegundaTela({super.key, required this.mensagem});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Segunda Tela')),
      body: Center(
        child: Text(mensagem), // Exibe o dado recebido
      ),
    );
  }
}
```

5.3. Rotas Nomeadas

Para aplicativos maiores, usar rotas nomeadas é mais organizado. Você define um mapa de nomes de rotas para construtores de Widgets no seu `MaterialApp`.

5.3.1. Configurando Rotas Nomeadas

No seu `MaterialApp` (geralmente no `main.dart`):

```
MaterialApp(  
  title: 'App com Rotas',  
  initialRoute: '/', // Rota inicial  
  routes: {  
    '/': (context) => const PrimeiraTela(),  
    '/segunda': (context) => const SegundaTela(),  
    '/terceira': (context) => const TerceiraTela(),  
  },  
);
```

5.3.2. Navegando com Rotas Nomeadas

Para navegar, use `Navigator.pushNamed()`:

```
// Navegando para a rota '/segunda'  
Navigator.pushNamed(context, '/segunda');
```

5.4. Navegação Avançada (Navigator 2.0 - Resumo)

O `Navigator 2.0` (também conhecido como *Router API*) é uma abordagem mais declarativa e complexa, projetada para lidar com cenários mais avançados, como:

- **Deep Linking:** Abrir o aplicativo em uma tela específica a partir de um link externo.
- **Navegação em Múltiplas Telas:** Gerenciar a pilha de telas em *tablets* ou *desktop*.

Para Iniciantes:

- Foque no `Navigator 1.0` (`push/pop` com `MaterialPageRoute` e Rotas Nomeadas).
- O `Navigator 2.0` é um tópico avançado que envolve `Router`, `RouterDelegate` e `RouteInformationParser`. Entender o `Navigator 1.0` é o primeiro passo essencial.

Característica	Navigator 1.0 (Imperativo)	Navigator 2.0 (Declarativo)
Estilo	Baseado em métodos (<code>push</code> , <code>pop</code>).	Baseado em estado (altera a lista de páginas).
Complexidade	Simples, ideal para iniciantes.	Complexo, ideal para cenários avançados.
Uso	Aplicativos simples e navegação sequencial.	Deep linking, navegação web, múltiplas telas.

5.5. Exercícios do Capítulo 5

1. Explique a analogia da “pilha de cartas” em relação à navegação em Flutter.
 2. Qual método do `Navigator` você usaria para ir para uma nova tela?
 3. Qual método do `Navigator` você usaria para fechar a tela atual e voltar para a anterior?
 4. Crie um trecho de código que configure duas rotas nomeadas: `/home` e `/perfil`.
 5. Como você passaria uma variável do tipo `int` chamada `userId` da tela atual para a próxima tela usando `MaterialPageRoute` ?
-

Próximo Capítulo: No Capítulo 6, você aprenderá a lidar com o estado de forma mais eficiente e escalável, introduzindo as bibliotecas de Gerenciamento de Estado.

Capítulo 6: Gerenciamento de Estado

Intermediário

No Capítulo 4, você aprendeu a usar o `setState()` para gerenciar o estado local de um `StatefulWidget`. No entanto, em aplicativos maiores, o estado precisa ser compartilhado entre muitos Widgets, muitas vezes distantes na árvore de Widgets. O `setState()` se torna inviável, levando ao problema conhecido como *Prop Drilling* (ter que passar o estado por muitos Widgets intermediários).

Para resolver isso, o Flutter adota o conceito de **Gerenciamento de Estado Global**.

6.1. O Problema do Estado Global

Quando o estado de um Widget afeta outro Widget distante, precisamos de uma maneira de:

1. **Armazenar** o estado em um local acessível a todos.
2. **Notificar** os Widgets interessados quando o estado muda.
3. **Reconstruir** apenas os Widgets que dependem daquele estado.

As soluções mais populares para isso são o **Provider** e o **Riverpod**.

6.2. Introdução ao Provider

O **Provider** é a solução de gerenciamento de estado mais popular e recomendada pelo próprio time do Flutter para iniciantes. Ele é baseado no conceito de *InheritedWidget* e *ChangeNotifier*.

6.2.1. O Conceito de `ChangeNotifier`

Um `ChangeNotifier` é uma classe que você cria para armazenar o estado e a lógica de negócios. Ele tem um método crucial: `notifyListeners()`.

```
import 'package:flutter/foundation.dart';

class ContadorModel extends ChangeNotifier {
  int _contador = 0;

  int get contador => _contador;

  void incrementar() {
    _contador++;
    // Notifica todos os Widgets que estão "ouvindo" este modelo
    notifyListeners();
  }
}
```

6.2.2. Expondo o Estado com Provider

Para que os Widgets possam acessar o `ContadorModel`, você precisa “fornecê-lo” (provide) na árvore de Widgets, geralmente no topo do seu aplicativo.

```
import 'package:provider/provider.dart';

void main() {
  runApp(
    // O ChangeNotifierProvider expõe o ContadorModel para todos os Widgets
    // abaixo
    ChangeNotifierProvider(
      create: (context) => ContadorModel(),
      child: const MyApp(),
    ),
  );
}
```

6.2.3. Consumindo o Estado

Existem duas maneiras principais de um Widget acessar o estado:

1. `Provider.of<T>(context)` (ou `context.watch<T>()`): O Widget *ouve* as mudanças e é reconstruído quando o estado muda.
2. `Provider.of<T>(context, listen: false)` (ou `context.read<T>()`): O Widget apenas *lê* o estado ou chama métodos, mas não é reconstruído.

Exemplo de Consumo:

```
import 'package:provider/provider.dart';

class MeuTexto extends StatelessWidget {
  const MeuTexto({super.key});

  @override
  Widget build(BuildContext context) {
    // 1. Acessa o modelo e "ouve" as mudanças
    final contadorModel = context.watch<ContadorModel>();

    return Text(
      'Contador: ${contadorModel.contador}',
      style: const TextStyle(fontSize: 30),
    );
  }
}

class MeuBotao extends StatelessWidget {
  const MeuBotao({super.key});

  @override
  Widget build(BuildContext context) {
    // 2. Acessa o modelo, mas não "ouve" (listen: false)
    final contadorModel = context.read<ContadorModel>();

    return ElevatedButton(
      onPressed: contadorModel.incrementar, // Chama o método
      child: const Text('Incrementar'),
    );
  }
}
```

6.3. Introdução ao Riverpod (Otimização do Provider)

O **Riverpod** é uma reescrita do Provider que resolve algumas de suas limitações, como a dependência do `BuildContext` para acessar o estado. Ele é mais seguro em termos de compilação e mais flexível.

Conceitos Chave do Riverpod:

- **Provider** : O objeto que armazena o estado (o “repositório”).

- **ConsumerWidget** : O Widget que usa o estado.
- **WidgetRef** : O objeto que permite que o Widget interaja com os **Providers**.

Exemplo Básico com Riverpod:

1. Definindo o Provider:

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

// Cria um Provider que armazena um inteiro (o estado)
final contadorProvider = StateProvider<int>((ref) => 0);
```

2. Usando o ProviderScope :

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

void main() {
  // O ProviderScope é o Widget que habilita o Riverpod
  runApp(const ProviderScope(child: MyApp()));
}
```

3. Consumindo o Estado:

```

class ContadorRiverpod extends ConsumerWidget {
  const ContadorRiverpod({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    // Observa o estado e reconstrói quando ele muda
    final contador = ref.watch(contadorProvider);

    return Column(
      children: [
        Text('Contador: $contador'),
        ElevatedButton(
          onPressed: () {
            // Altera o estado
            ref.read(contadorProvider.notifier).state++;
          },
          child: const Text('Incrementar'),
        ),
      ],
    );
  }
}

```

Recomendação para Iniciantes:

- Comece com o **Provider**. Ele é mais simples de entender e suficiente para a maioria dos aplicativos de médio porte.
- O **Riverpod** é uma excelente evolução, mas pode ser introduzido após você dominar o conceito de estado global com o Provider.

6.4. Exercícios do Capítulo 6

1. Qual problema o Gerenciamento de Estado Global busca resolver em aplicativos Flutter?
2. Qual é a principal função do método `notifyListeners()` em uma classe `ChangeNotifier` do Provider?
3. Qual é a diferença prática entre usar `context.watch<T>()` e `context.read<T>()` ao consumir um `Provider` ?

4. Qual Widget é necessário para habilitar o uso do Riverpod em um aplicativo?
5. Crie uma classe `TemaModel` que herde de `ChangeNotifier` e tenha uma propriedade booleana `isDarkTheme`. Adicione um método `toggleTheme()` que alterne o valor e notifique os ouvintes.

Próximo Capítulo: No Capítulo 7, você aprenderá a lidar com coleções de dados e a exibi-las de forma eficiente usando `ListView`.

Capítulo 7: Listas e Dados Dinâmicos

A maioria dos aplicativos móveis lida com coleções de dados, como listas de e-mails, feeds de notícias ou listas de tarefas. O Flutter oferece Widgets poderosos para exibir esses dados de forma eficiente: o `ListView`.

7.1. O Problema da Exibição de Listas

Se você tem uma lista de 1000 itens, criar 1000 Widgets `Text` dentro de um `Column` seria ineficiente e consumiria muita memória. O `ListView` resolve isso com o conceito de **Virtualização**.

7.2. O Widget `ListView`

O `ListView` é o Widget mais comum para exibir listas de itens roláveis. Ele só constrói os Widgets que estão visíveis na tela, economizando recursos.

Existem três construtores principais do `ListView`:

7.2.1. `ListView` (Construtor Padrão)

Usado para listas pequenas e estáticas, onde todos os itens são conhecidos e construídos de uma vez.

```

ListView(
  padding: const EdgeInsets.all(8),
  children: <Widget>[
    Container(height: 50, color: Colors.red, child: const Center(child:
Text('Item 1'))),
    Container(height: 50, color: Colors.green, child: const Center(child:
Text('Item 2'))),
    Container(height: 50, color: Colors.blue, child: const Center(child:
Text('Item 3'))),
  ],
)

```

7.2.2. `ListView.builder` (O Mais Usado)

Este é o construtor ideal para listas grandes ou infinitas. Ele constrói os itens **sob demanda** (apenas quando eles estão prestes a aparecer na tela).

Propriedade	Descrição
<code>itemCount</code>	O número total de itens na lista.
<code>itemBuilder</code>	Uma função que é chamada para construir cada item da lista. Recebe o <code>BuildContext</code> e o <code>index</code> (posição do item).

Exemplo de `ListView.builder` :

```

final List<String> nomes = ['Alice', 'Bob', 'Charlie', 'David', 'Eve'];

ListView.builder(
  itemCount: nomes.length,
  itemBuilder: (BuildContext context, int index) {
    return ListTile(
      leading: const Icon(Icons.person),
      title: Text(nomes[index]),
      subtitle: Text('Posição: $index'),
      onTap: () {
        // Ação ao clicar no item
      },
    );
  },
)

```


7.2.3. `ListView.separated`

Semelhante ao `builder`, mas permite adicionar um Widget separador entre cada item.

```
ListView.separated(  
  itemCount: nomes.length,  
  itemBuilder: (context, index) {  
    return ListTile(title: Text(nomes[index]));  
  },  
  separatorBuilder: (context, index) {  
    // Adiciona um divisor cinza entre os itens  
    return const Divider(color: Colors.grey);  
  },  
)
```

7.3. O Widget `ListTile`

O `ListTile` é um Widget de conveniência projetado especificamente para ser usado em listas. Ele facilita a criação de linhas com um ícone à esquerda (`leading`), um título (`title`), um subtítulo (`subtitle`) e um Widget à direita (`trailing`).

7.4. Exemplo Prático: Lista de Tarefas (Parte 1)

Vamos começar a construir o aplicativo de lista de tarefas, focando apenas na exibição da lista.

```

import 'package:flutter/material.dart';

class ListaTarefasPage extends StatefulWidget {
  const ListaTarefasPage({super.key});

  @override
  State<ListaTarefasPage> createState() => _ListaTarefasPageState();
}

class _ListaTarefasPageState extends State<ListaTarefasPage> {
  // Lista de tarefas (o estado)
  final List<String> _tarefas = ['Comprar leite', 'Estudar Flutter', 'Fazer
exercícios'];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Minhas Tarefas'),
      ),
      body: ListView.builder(
        itemCount: _tarefas.length,
        itemBuilder: (context, index) {
          return ListTile(
            title: Text(_tarefas[index]),
            trailing: IconButton(
              icon: const Icon(Icons.delete, color: Colors.red),
              onPressed: () {
                // Ação de remover tarefa (será implementada no próximo
capítulo)
              },
            ),
          );
        },
      ),
    );
  }
}

```

7.5. Exercícios do Capítulo 7

1. Qual é a principal vantagem de usar `ListView.builder` em vez do construtor padrão `ListView` para listas muito longas?

- 2. Qual propriedade do `ListView.builder` é responsável por dizer ao Flutter quantos itens ele deve construir?
- 3. Qual é a função do `itemBuilder` no `ListView.builder` ?
- 4. Crie um `ListView.separated` que exiba 5 cores diferentes (ex: Vermelho, Azul, Verde, Amarelo, Roxo) e use um `Divider` como separador.
- 5. O que é o Widget `ListTile` e quais são suas principais propriedades?

Próximo Capítulo: No Capítulo 8, você aprenderá a capturar a entrada do usuário usando formulários e a validar esses dados.

Capítulo 8: Interação com o Usuário

A interação com o usuário é fundamental em qualquer aplicativo. Neste capítulo, focaremos em como capturar dados de entrada usando formulários, como validar esses dados e como gerenciar o texto digitado.

8.1. O Widget `TextField`

O `TextField` é o Widget básico para entrada de texto. Ele permite que o usuário digite texto usando o teclado virtual ou físico.

Propriedades Chave:

Propriedade	Descrição
<code>decoration</code>	Um <code>InputDecoration</code> que define a aparência (rótulo, dica, ícone, borda).
<code>keyboardType</code>	Define o tipo de teclado a ser exibido (ex: <code>TextInputType.number</code> para números).
<code>obscureText</code>	Se <code>true</code> , oculta o texto (ideal para senhas).
<code>onChanged</code>	Função chamada a cada alteração no texto.
<code>onSubmitted</code>	Função chamada quando o usuário pressiona “Enter” ou “Concluído” .

Exemplo Básico:

```
TextField(  
  decoration: const InputDecoration(  
    labelText: 'Nome de Usuário',  
    hintText: 'Digite seu nome',  
    border: OutlineInputBorder(),  
    icon: Icon(Icons.person),  
  ),  
  keyboardType: TextInputType.text,  
)
```

8.2. Gerenciando o Texto com `TextEditingController`

Para ler, modificar ou reagir ao texto de um `TextField` de forma programática, você deve usar um `TextEditingController`.

1. **Criação:** Declare o `Controller` no estado do seu `StatefulWidget`.
2. **Associação:** Associe-o ao `TextField` através da propriedade `controller`.
3. **Uso:** Acesse o texto usando `_controller.text`.

```

class MeuFormulario extends StatefulWidget {
  const MeuFormulario({super.key});

  @override
  State<MeuFormulario> createState() => _MeuFormularioState();
}

class _MeuFormularioState extends State<MeuFormulario> {
  // 1. Criação do Controller
  final TextEditingController _nomeController = TextEditingController();

  @override
  void dispose() {
    // 4. É importante descartar o controller quando o Widget for removido
    _nomeController.dispose();
    super.dispose();
  }

  void _exibirNome() {
    // 3. Uso do Controller
    print('O nome digitado é: ${_nomeController.text}');
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        TextField(
          // 2. Associação
          controller: _nomeController,
          decoration: const InputDecoration(labelText: 'Seu Nome'),
        ),
        ElevatedButton(
          onPressed: _exibirNome,
          child: const Text('Exibir'),
        ),
      ],
    );
  }
}

```

8.3. Validação de Formulários

Para validar múltiplos campos de entrada e garantir que os dados estejam corretos antes de serem processados, usamos os Widgets `Form` e `TextFormField`.

8.3.1. `Form` e `GlobalKey`

O Widget `Form` atua como um contêiner para agrupar vários `TextFormField`. Para interagir com o `Form` (como validar ou salvar), você precisa de uma `GlobalKey`.

```
// 1. Crie a chave no estado do seu StatefulWidget
final _formKey = GlobalKey<FormState>();

// ... dentro do método build()
Form(
  key: _formKey, // 2. Associe a chave ao Form
  child: Column(
    // ... seus TextFormFields
  ),
)
```

8.3.2. `TextFormField`

O `TextFormField` é uma versão do `TextField` que se integra ao `Form` e possui a propriedade `validator`.

A Propriedade `validator`:

É uma função que recebe o valor do campo e retorna:

- `null`: Se a validação for bem-sucedida.
- Uma `String`: Se a validação falhar (a string é a mensagem de erro a ser exibida).

Exemplo de Validação:

```

TextFormField(
  decoration: const InputDecoration(labelText: 'E-mail'),
  validator: (value) {
    if (value == null || value.isEmpty) {
      return 'Por favor, insira seu e-mail';
    }
    if (!value.contains('@')) {
      return 'E-mail inválido';
    }
    return null; // Validação bem-sucedida
  },
)

```

8.3.3. Disparando a Validação

Para validar o formulário inteiro, você chama o método `validate()` na `currentState` da sua `GlobalKey`.

```

ElevatedButton(
  onPressed: () {
    // Valida todos os TextFormFields dentro do Form
    if (_formKey.currentState!.validate()) {
      // Se for válido, exibe uma mensagem ou envia os dados
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Processando Dados')),
      );
    }
  },
  child: const Text('Enviar'),
)

```

8.4. Exercícios do Capítulo 8

1. Qual Widget é o principal responsável pela entrada de texto em Flutter?
2. Qual é a finalidade do `TextEditingController` ?
3. Qual método você deve chamar no `TextEditingController` quando o Widget for descartado (`dispose`)?
4. Qual Widget atua como um contêiner para agrupar e validar múltiplos campos de entrada?

5. Crie um `TextFormField` que valide se o campo “Senha” tem pelo menos 6 caracteres. Se não tiver, a mensagem de erro deve ser “A senha deve ter no mínimo 6 caracteres” .

Próximo Capítulo: No Capítulo 9, você aprenderá a fazer seu aplicativo se comunicar com serviços externos, consumindo dados de uma API REST.

Capítulo 9: Comunicação com o Mundo Exterior

Um aplicativo moderno raramente vive isolado. A maioria precisa se comunicar com servidores para buscar ou enviar dados. Isso é feito através de **APIs (Application Programming Interfaces)**, sendo as APIs **REST (Representational State Transfer)** as mais comuns.

9.1. O Pacote `http`

O Flutter não possui um cliente HTTP nativo no seu *core*. O pacote mais simples e popular para fazer requisições HTTP é o `http` .

9.1.1. Adicionando a Dependência

Antes de usar, você deve adicionar o pacote `http` ao seu arquivo `pubspec.yaml` :

```
dependencies:  
  flutter:  
    sdk: flutter  
  http: ^1.1.0 # Use a versão mais recente
```

Após salvar, execute `flutter pub get` no terminal.

9.1.2. Fazendo uma Requisição GET

A requisição `GET` é usada para buscar dados de um servidor. Usaremos a API pública [JSONPlaceholder](#) para buscar uma lista de posts.

```
import 'dart:convert'; // Para decodificar JSON
import 'package:http/http.dart' as http; // Importa o pacote http

Future<void> buscarPosts() async {
  // 1. Defina a URL
  final url = Uri.parse('https://jsonplaceholder.typicode.com/posts');

  // 2. Faça a requisição
  final response = await http.get(url);

  // 3. Verifique o status da resposta
  if (response.statusCode == 200) {
    // 4. Decodifique o corpo da resposta (JSON)
    final List<dynamic> jsonResponse = jsonDecode(response.body);

    // Exemplo: Imprime o título do primeiro post
    print('Título do primeiro post: ${jsonResponse[0]['title']}');
  } else {
    // Se a requisição falhar
    print('Falha ao carregar posts. Status: ${response.statusCode}');
  }
}
```

9.2. Lidando com Dados: Serialização JSON

Os dados que vêm do servidor geralmente estão no formato **JSON (JavaScript Object Notation)**. Para usá-los no Dart, precisamos convertê-los (ou *serializá-los*) em objetos Dart.

9.2.1. Criando um Modelo (Model)

É uma boa prática criar uma classe Dart que represente a estrutura do JSON.

```

class Post {
    final int userId;
    final int id;
    final String title;
    final String body;

    Post({
        required this.userId,
        required this.id,
        required this.title,
        required this.body,
    });

    // Construtor de fábrica para criar um Post a partir de um mapa JSON
    factory Post.fromJson(Map<String, dynamic> json) {
        return Post(
            userId: json['userId'],
            id: json['id'],
            title: json['title'],
            body: json['body'],
        );
    }
}

```

9.2.2. Usando o Modelo na Requisição

```

Future<List<Post>> buscarPostsComModelo() async {
    final url = Uri.parse('https://jsonplaceholder.typicode.com/posts');
    final response = await http.get(url);

    if (response.statusCode == 200) {
        final List<dynamic> jsonResponse = jsonDecode(response.body);

        // Mapeia a lista de JSONs para uma lista de objetos Post
        return jsonResponse.map((json) => Post.fromJson(json)).toList();
    } else {
        throw Exception('Falha ao carregar posts');
    }
}

```

9.3. Exibindo Dados Assíncronos com `FutureBuilder`

Requisições de rede são **assíncronas** (levam tempo para serem concluídas). O Widget `FutureBuilder` é perfeito para lidar com isso, pois ele reconstrói a interface automaticamente quando o `Future` (o resultado da requisição) é concluído.

Estado do <code>Future</code>	O que o <code>FutureBuilder</code> faz
<code>ConnectionState.waiting</code>	Exibe um indicador de carregamento (<code>CircularProgressIndicator</code>).
<code>ConnectionState.done</code>	Se houver dados, exibe os dados. Se houver erro, exibe a mensagem de erro.

9.4. Exemplo Prático: App que Consome uma API

Pública

```
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';

// Modelo Post (como definido acima)
class Post {
  final int id;
  final String title;
  final String body;
  Post({required this.id, required this.title, required this.body, required int
userId});
  factory Post.fromJson(Map<String, dynamic> json) {
    return Post(
      userId: json['userId'],
      id: json['id'],
      title: json['title'],
      body: json['body'],
    );
  }
}

// Função de busca (como definida acima)
Future<List<Post>> fetchPosts() async {
  final response = await
http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts'));
  if (response.statusCode == 200) {
    final List<dynamic> jsonResponse = jsonDecode(response.body);
    return jsonResponse.map((json) => Post.fromJson(json)).toList();
  } else {
    throw Exception('Falha ao carregar posts');
  }
}

class ApiConsumerApp extends StatelessWidget {
  const ApiConsumerApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Consumo de API',
      home: Scaffold(
        appBar: AppBar(title: const Text('Posts da API')),
        body: Center(
```

```

child: FutureBuilder<List<Post>>(
  future: fetchPosts(), // O Future a ser observado
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      // Enquanto espera, mostra um indicador de progresso
      return const CircularProgressIndicator();
    } else if (snapshot.hasError) {
      // Se houver erro
      return Text('Erro: ${snapshot.error}');
    } else if (snapshot.hasData) {
      // Se os dados foram carregados com sucesso
      return ListView.builder(
        itemCount: snapshot.data!.length,
        itemBuilder: (context, index) {
          final post = snapshot.data![index];
          return ListTile(
            title: Text(post.title),
            subtitle: Text(post.body, maxLines: 2, overflow:
TextOverflow.ellipsis),
          );
        },
      );
    }
    // Caso padrão (nunca deve acontecer)
    return const Text('Nenhum dado encontrado');
  },
),
),
),
);
}
}

```

9.5. Exercícios do Capítulo 9

1. Qual pacote é comumente usado em Flutter para fazer requisições HTTP?
2. Qual é a finalidade do `dart:convert` e qual função dele é usada para transformar uma string JSON em um mapa Dart?
3. O que é um `Future` e por que as requisições de rede em Flutter retornam um `Future`?

4. Qual Widget é ideal para exibir o estado de um `Future` (carregando, erro, sucesso) na interface?
5. Modifique o `fetchPosts` para que ele busque apenas um único post (ex: `https://jsonplaceholder.typicode.com/posts/1`) e retorne um único objeto `Post` em vez de uma lista.

Próximo Capítulo: No Capítulo 10, você aprenderá a armazenar dados simples diretamente no dispositivo do usuário.

Capítulo 10: Persistência de Dados Simples

Em muitos casos, você precisa armazenar dados simples no dispositivo do usuário, como configurações de tema (claro/escuro), a última vez que o aplicativo foi aberto ou o estado de um *checkbox*. Para isso, o Flutter oferece o pacote `shared_preferences`.

10.1. O Pacote `shared_preferences`

O `shared_preferences` é um pacote que fornece uma maneira persistente e simples de armazenar dados primitivos (inteiros, duplos, booleanos, strings e listas de strings) em pares chave-valor.

- **No Android:** Usa `SharedPreferences`.
- **No iOS:** Usa `NSUserDefaults`.

10.1.1. Adicionando a Dependência

Adicione o pacote ao seu `pubspec.yaml`:

```
dependencies:  
  flutter:  
    sdk: flutter  
  shared_preferences: ^2.2.2 # Use a versão mais recente
```

Execute `flutter pub get`.

10.2. Armazenando e Lendo Dados

O uso do `shared_preferences` é baseado em dois passos principais: obter a instância e, em seguida, usar os métodos de leitura e escrita.

10.2.1. Obtendo a Instância

A obtenção da instância é um processo assíncrono:

```
import 'package:shared_preferences/shared_preferences.dart';  
  
Future<SharedPreferences> getPrefs() async {  
  return await SharedPreferences.getInstance();  
}
```

10.2.2. Escrevendo Dados

Os métodos de escrita são prefixados com `set` e são assíncronos:

```
Future<void> salvarTema(bool isDark) async {  
  final prefs = await SharedPreferences.getInstance();  
  // Salva um booleano com a chave 'isDarkTheme'  
  await prefs.setBool('isDarkTheme', isDark);  
  print('Tema salvo: $isDark');  
}
```

10.2.3. Lendo Dados

Os métodos de leitura são prefixados com `get` e são síncronos, mas a chamada para `getInstance()` é assíncrona:


```
Future<bool> carregarTema() async {  
  final prefs = await SharedPreferences.getInstance();  
  // Tenta obter o booleano. Se não existir, retorna false (valor padrão)  
  final isDark = prefs.getBool('isDarkTheme') ?? false;  
  return isDark;  
}
```

Métodos de Leitura e Escrita:

Tipo de Dado	Método de Escrita	Método de Leitura
String	<code>setString(key, value)</code>	<code>getString(key)</code>
int	<code>setInt(key, value)</code>	<code>getInt(key)</code>
bool	<code>setBool(key, value)</code>	<code>getBool(key)</code>
double	<code>setDouble(key, value)</code>	<code>getDouble(key)</code>
List	<code>setStringList(key, value)</code>	<code>getStringList(key)</code>

10.3. Exemplo Prático: Persistindo o Tema

Vamos usar o `shared_preferences` para persistir a escolha do tema (claro/escuro) do usuário.

```

import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

class TemaPage extends StatefulWidget {
  const TemaPage({super.key});

  @override
  State<TemaPage> createState() => _TemaPageState();
}

class _TemaPageState extends State<TemaPage> {
  bool _isDarkTheme = false;

  @override
  void initState() {
    super.initState();
    _carregarTema();
  }

  // Carrega o tema salvo ao iniciar a tela
  void _carregarTema() async {
    final prefs = await SharedPreferences.getInstance();
    setState(() {
      _isDarkTheme = prefs.getBool('isDarkTheme') ?? false;
    });
  }

  // Alterna o tema e salva a preferência
  void _alternarTema(bool value) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool('isDarkTheme', value);
    setState(() {
      _isDarkTheme = value;
    });
  }

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      theme: _isDarkTheme ? ThemeData.dark() : ThemeData.light(),
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Configurações de Tema'),
        ),
        body: Center(
          child: SwitchListTile(

```

```
        title: const Text('Tema Escuro'),
        value: _isDarkTheme,
        onChanged: _alternarTema,
      ),
    ),
  ),
);
}
```

10.4. Exercícios do Capítulo 10

1. Qual é a principal limitação do `shared_preferences` em termos de tipos de dados que ele pode armazenar?
2. Qual método você deve chamar para obter a instância do `SharedPreferences`?
3. Crie uma função que salve um `int` chamado `contadorCliques` com o valor 10.
4. Crie uma função que carregue o valor de `contadorCliques` e retorne 0 se a chave não existir.
5. Qual pacote nativo o `shared_preferences` utiliza no iOS?

Próximo Capítulo: No Capítulo 11, focaremos em como escrever código limpo e sustentável, abordando as Boas Práticas de Código.

Capítulo 11: Boas Práticas e Arquitetura

Escrever código que funciona é apenas metade da batalha. A outra metade é escrever código que seja **legível, manutenível e escalável**. Este capítulo aborda as boas práticas essenciais no desenvolvimento Flutter.

11.1. Organização de Pastas (Arquitetura Simples)

Uma estrutura de pastas bem definida é crucial para a manutenibilidade do projeto. A seguir, uma estrutura simples e recomendada para iniciantes, focada na separação de

responsabilidades:

```
lib/
├─ main.dart           # Ponto de entrada do aplicativo
├─ models/             # Classes de dados (Modelos de API, objetos de
domínio)
|   └─ post.dart
├─ pages/              # Telas completas (Widgets que herdam de Scaffold)
|   └─ home_page.dart
|   └─ login_page.dart
├─ widgets/            # Componentes de UI reutilizáveis (Botões, Cards,
etc.)
|   └─ custom_button.dart
|   └─ post_card.dart
├─ services/           # Lógica de comunicação com API, banco de dados,
etc.
|   └─ api_service.dart
└─ providers/          # Classes de gerenciamento de estado
(ChangeNotifier, Riverpod Providers)
    └─ counter_provider.dart
```

Princípio: Separe o código por **função** (o que ele faz), não por tipo (Stateless/Stateful).

11.2. Boas Práticas de Nomenclatura (Convenções Dart)

O Dart possui convenções de nomenclatura que devem ser seguidas para manter a consistência do código.

Elemento	Convenção	Exemplo
Arquivos	<code>snake_case</code> (minúsculas com sublinhado)	<code>user_profile_page.dart</code>
Classes	<code>PascalCase</code> (primeira letra de cada palavra maiúscula)	<code>UserProfilePage</code> , <code>PostModel</code>
Funções/Variáveis	<code>camelCase</code> (primeira letra minúscula, demais palavras maiúsculas)	<code>fetchData</code> , <code>userName</code> , <code>_privateVariable</code>
Constantes	<code>SCREAMING_SNAKE_CASE</code> (raramente usado, exceto para constantes globais)	<code>MAX_LENGTH</code>

Dica de Ouro: Use o prefixo `_` (sublinhado) para tornar uma variável, função ou classe **privada** ao arquivo onde foi definida.

```
// Variável privada ao arquivo
int _contador = 0;

// Função privada ao arquivo
void _incrementar() {
  _contador++;
}
```

11.3. O Princípio DRY (Don't Repeat Yourself)

Se você se encontrar copiando e colando o mesmo bloco de código mais de uma vez, é um sinal de que você deve extraí-lo para um Widget ou função reutilizável.

Exemplo: Se você usa o mesmo estilo de `Container` em várias telas, crie um Widget chamado `CustomCard` na pasta `widgets/`.

11.4. Otimização e Performance

11.4.1. Use `const` Sempre que Possível

O uso da palavra-chave `const` (constante) é a otimização de performance mais fácil e importante no Flutter.

- **O que faz:** Diz ao Flutter para criar o Widget apenas uma vez e reutilizá-lo, em vez de reconstruí-lo a cada chamada do `build()`.
- **Onde usar:** Em qualquer Widget que não mude (ex: `const Text('Título')`, `const Padding(...)`).

11.4.2. Minimize o Uso de `setState()`

O `setState()` reconstrói todo o `StatefulWidget`. Em Widgets grandes, isso pode ser lento.

- **Solução:** Mantenha o `StatefulWidget` o menor possível. Mova a lógica de estado para um gerenciador de estado (Provider/Riverpod) e use `setState()` apenas para estados locais e triviais.

11.5. Dicas para Debug

1. **Use o `print()` com moderação:** Para debug rápido, `print()` funciona, mas polui o console.
2. **Use o Debugger da IDE:** Aprenda a usar *breakpoints* (pontos de parada) no VS Code ou Android Studio. Isso permite inspecionar o valor das variáveis em tempo real.
3. **Use o Flutter DevTools:** Uma suíte de ferramentas poderosa que permite inspecionar a árvore de Widgets, monitorar a performance e o uso de memória. Execute `flutter pub global activate devtools` e depois `flutter pub global run devtools` no terminal.
4. **Verifique a Árvore de Widgets:** Se um Widget não está aparecendo, use o DevTools para inspecionar a árvore e verificar se ele está sendo construído no local e com as restrições de tamanho corretas.

11.6. Sugestões de Bibliotecas Úteis

Biblioteca	Propósito	Por que é útil?
<code>url_launcher</code>	Abrir URLs no navegador ou aplicativos externos (e-mail, telefone).	Essencial para links externos.
<code>intl</code>	Internacionalização e formatação de datas, moedas e números.	Para aplicativos globais.
<code>cached_network_image</code>	Exibir imagens da internet com cache.	Melhora a performance e economiza dados.
<code>flutter_slidable</code>	Adicionar ações de deslizar em itens de lista (como no Gmail).	Melhora a experiência do usuário.

11.7. Exercícios do Capítulo 11

1. Qual é o principal princípio por trás da organização de pastas recomendada?
2. Qual convenção de nomenclatura você usaria para o nome de uma classe chamada “Lista de Produtos” ?
3. Qual é o efeito da palavra-chave `const` em um Widget?
4. Se você precisa depurar um problema de performance, qual ferramenta do Flutter você deve usar?
5. O que significa o prefixo `_` em uma variável Dart?

Próximo Capítulo: No Capítulo 12, você terá um glossário completo dos termos mais usados em Flutter. Em seguida, faremos a revisão final e a entrega.

Capítulo 12: Glossário de Termos

Este glossário contém os termos mais comuns e importantes que você encontrará ao longo de sua jornada com Flutter.

Termo	Definição
API	Application Programming Interface. Conjunto de regras e protocolos que permite que diferentes softwares se comuniquem.
Asset	Arquivos estáticos (imagens, fontes, JSONs) que são empacotados com o aplicativo.
BuildContext	Um “handle” para a localização de um Widget na árvore de Widgets. É usado para acessar temas, navegadores e provedores.
Dart	A linguagem de programação otimizada para UI, desenvolvida pelo Google e usada pelo Flutter.
Deep Linking	A capacidade de um link externo (como um URL) abrir o aplicativo em uma tela específica.
Dependency	Uma biblioteca ou pacote externo que seu projeto utiliza (listado no <code>pubspec.yaml</code>).
DevTools	Um conjunto de ferramentas de depuração e inspeção para aplicativos Dart e Flutter.
Future	Um objeto que representa um valor ou erro que estará disponível em algum momento no futuro (usado em operações assíncronas).
Hot Reload	Recurso do Flutter que injeta o código-fonte atualizado em um aplicativo em execução, permitindo ver as mudanças quase instantaneamente.
InheritedWidget	Um tipo especial de Widget que permite que seus descendentes acessem dados de forma eficiente. Base para o Provider.
JSON	JavaScript Object Notation. Formato de texto leve e legível por humanos para troca de dados.
Key	Um identificador opcional usado para preservar o estado de um Widget quando ele se move na árvore de Widgets.
Material Design	Um sistema de design criado pelo Google que fornece diretrizes para a aparência e o comportamento da interface do usuário.
Navigator	O Widget que gerencia a pilha de telas (rotas) do aplicativo.
Package/Pub	Um pacote de código Dart ou Flutter que pode ser importado e usado em seu projeto. O repositório oficial é o pub.dev .

Termo	Definição
Provider	Uma biblioteca popular de gerenciamento de estado baseada em <code>InheritedWidget</code> e <code>ChangeNotifier</code> .
Route	Uma abstração para uma tela ou página do aplicativo.
Scaffold	O Widget que fornece a estrutura básica para uma tela de aplicativo (AppBar, Body, FloatingActionButton).
SDK	Software Development Kit. O conjunto de ferramentas que permite criar e compilar aplicativos Flutter.
State	Os dados que podem mudar durante a vida útil de um Widget e que afetam a aparência da interface.
StatefulWidget	Um Widget que pode ter estado mutável.
StatelessWidget	Um Widget que não tem estado mutável.
Widget	O bloco de construção fundamental do Flutter. Tudo na interface é um Widget.
Widget Tree	A hierarquia de Widgets que compõem a interface do usuário.

Conclusão: Você concluiu a parte teórica da apostila! No próximo passo, uniremos todos os capítulos em um único documento e adicionaremos os exemplos práticos completos e os exercícios finais.

Apêndice A: Exemplo Prático - App Contador

Este é o código completo do App Contador, que ilustra o uso de `StatefulWidget` e `setState()`, conforme discutido no Capítulo 4.

A.1. Código Completo (lib/main.dart)

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'App Contador Flutter',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: const ContadorPage(title: 'Contador Simples'),
    );
  }
}

// O Widget principal que contém o estado
class ContadorPage extends StatefulWidget {
  const ContadorPage({super.key, required this.title});

  final String title;

  @override
  State<ContadorPage> createState() => _ContadorPageState();
}

// A classe State que armazena e gerencia o estado
class _ContadorPageState extends State<ContadorPage> {
  // 1. O estado mutável
  int _contador = 0;

  // 2. Função que altera o estado e notifica o Flutter
  void _incrementarContador() {
    setState(() {
      // Altera o estado
      _contador++;
    });
  }
}
```

```

// Função para o exercício: Decrementar
void _decrementarContador() {
  setState(() {
    _contador--;
  });
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text(widget.title),
    ),
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: <Widget>[
          const Text(
            'Valor atual do contador:',
          ),
          Text(
            // 3. Exibe o estado
            '$_contador',
            style: Theme.of(context).textTheme.headlineLarge,
          ),
        ],
      ),
    ),
    // Botões de ação flutuantes
    floatingActionButton: Row(
      mainAxisAlignment: MainAxisAlignment.end,
      children: [
        FloatingActionButton(
          heroTag: "btn1", // Necessário para múltiplos FABs
          onPressed: _decrementarContador,
          tooltip: 'Decrementar',
          child: const Icon(Icons.remove),
        ),
        const SizedBox(width: 10),
        FloatingActionButton(
          heroTag: "btn2", // Necessário para múltiplos FABs
          onPressed: _incrementarContador,
          tooltip: 'Incrementar',
          child: const Icon(Icons.add),
        ),
      ],
    ),
  );
}

```

```

    ],
  ),
);
}
}

```

A.2. Explicação Linha por Linha

Linha(s)	Código	Explicação
1-3	<code>void main() { runApp(const MyApp()); }</code>	Ponto de entrada do aplicativo. Inicia o Widget raiz.
12	<code>class ContadorPage extends StatefulWidget {</code>	Define o Widget que terá seu estado gerenciado.
20	<code>class _ContadorPageState extends State<ContadorPage> {</code>	A classe que armazena o estado e a lógica de construção.
22	<code>int _contador = 0;</code>	A variável de estado que será alterada. O <code>_</code> a torna privada.
25-29	<code>void _incrementarContador() { setState(() { _contador++; }); }</code>	A função que incrementa o contador. O <code>setState()</code> é crucial para notificar o Flutter sobre a mudança.
39	<code>body: Center(...)</code>	O conteúdo principal da tela, centralizado.
47	<code>'\$_contador',</code>	Interpolação de string para exibir o valor atual da variável de estado.
58	<code>onPressed: _incrementarContador,</code>	Associa a função de incremento ao evento de clique do botão.
60	<code>child: const Icon(Icons.add),</code>	O ícone de adição no botão.

Próximo Apêndice: App de Lista de Tarefas.

Apêndice B: Exemplo Prático - App de Lista de Tarefas (To-Do List)

Este exemplo combina conceitos de `StatefulWidget`, `ListView.builder`, `TextEditingController` e `shared_preferences` para criar um aplicativo funcional de lista de tarefas.

B.1. Código Completo (lib/main.dart)

```
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'dart:convert'; // Para codificar/decodificar a lista

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Lista de Tarefas',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: const TodoListPage(),
    );
  }
}

class TodoListPage extends StatefulWidget {
  const TodoListPage({super.key});

  @override
  State<TodoListPage> createState() => _TodoListPageState();
}

class _TodoListPageState extends State<TodoListPage> {
  // Lista de tarefas (o estado)
  List<String> _tarefas = [];

  // Controller para o campo de texto
  final TextEditingController _controller = TextEditingController();

  @override
  void initState() {
    super.initState();
    _carregarTarefas(); // Carrega as tarefas salvas ao iniciar
  }
}
```

```

@Override
void dispose() {
    _controller.dispose();
    super.dispose();
}

// --- Métodos de Persistência (Capítulo 10) ---

void _salvarTarefas() async {
    final prefs = await SharedPreferences.getInstance();
    // Converte a lista de strings para uma string JSON e salva
    prefs.setString('tarefas', jsonEncode(_tarefas));
}

void _carregarTarefas() async {
    final prefs = await SharedPreferences.getInstance();
    final tarefasSalvas = prefs.getString('tarefas');
    if (tarefasSalvas != null) {
        // Decodifica a string JSON de volta para uma lista de strings
        setState(() {
            _tarefas = List<String>.from(jsonDecode(tarefasSalvas));
        });
    }
}

// --- Métodos de Gerenciamento de Estado (Capítulo 4) ---

void _adicionarTarefa() {
    if (_controller.text.isNotEmpty) {
        setState(() {
            _tarefas.add(_controller.text);
            _controller.clear(); // Limpa o campo de texto
            _salvarTarefas(); // Salva a nova lista
        });
    }
}

void _removerTarefa(int index) {
    setState(() {
        _tarefas.removeAt(index);
        _salvarTarefas(); // Salva a lista atualizada
    });
}

// --- Construção da Interface (Capítulos 3 e 7) ---

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Minha Lista de Tarefas'),
    ),
    body: Column(
      children: <Widget>[
        Padding(
          padding: const EdgeInsets.all(8.0),
          child: Row(
            children: <Widget>[
              Expanded(
                child: TextField(
                  controller: _controller,
                  decoration: const InputDecoration(
                    labelText: 'Nova Tarefa',
                    hintText: 'Ex: Comprar pão',
                    border: OutlineInputBorder(),
                  ),
                  onSubmitted: (_) => _adicionarTarefa(), // Adiciona ao
pressionar Enter
                ),
              const SizedBox(width: 10),
              ElevatedButton(
                onPressed: _adicionarTarefa,
                style: ElevatedButton.styleFrom(
                  padding: const EdgeInsets.all(15),
                ),
                child: const Text('Adicionar'),
              ),
            ],
          ),
        ),
        Expanded(
          // Lista de tarefas rolável
          child: ListView.builder(
            itemCount: _tarefas.length,
            itemBuilder: (context, index) {
              return ListTile(
                title: Text(_tarefas[index]),
                trailing: IconButton(
                  icon: const Icon(Icons.delete, color: Colors.red),
                  onPressed: () => _removerTarefa(index), // Chama a função
de remoção

```



```

        ),
      ),
    },
  ),
),
],
),
);
}
}

```

B.2. Explicação dos Conceitos Integrados

Conceito	Onde é Usado	Explicação
StatefulWidget	<code>TodoListPage</code> e <code>_TodoListPageState</code>	Permite que a lista de tarefas (<code>_tarefas</code>) seja alterada e a interface seja reconstruída.
TextEditingController	<code>_controller</code>	Gerencia o texto digitado no <code>TextField</code> para adicionar novas tarefas.
ListView.builder	<code>Expanded(child: ListView.builder(...))</code>	Exibe a lista de tarefas de forma eficiente, construindo os <code>ListTile</code> sob demanda.
setState()	<code>_adicionarTarefa()</code> e <code>_removerTarefa()</code>	Notifica o Flutter para reconstruir a interface após a lista <code>_tarefas</code> ser modificada.
shared_preferences	<code>_salvarTarefas()</code> e <code>_carregarTarefas()</code>	Garante que as tarefas permaneçam salvas mesmo após o aplicativo ser fechado e reaberto.
jsonEncode / jsonDecode	Métodos de persistência	Usados para converter a lista de Dart em uma string JSON para salvar e vice-versa.

Próximo Apêndice: App que Consome uma API Pública.

Apêndice C: Exemplo Prático - App que Consome uma API Pública

Este exemplo demonstra como buscar dados de uma API REST usando o pacote `http` e como exibir o estado assíncrono (carregando, erro, sucesso) usando o `FutureBuilder`, conforme discutido no Capítulo 9.

C.1. Código Completo (`lib/main.dart`)

Observação: Certifique-se de ter adicionado a dependência `http: ^1.1.0` no seu `pubspec.yaml` e executado `flutter pub get`.

```

import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Consumidor de API',
      theme: ThemeData(
        primarySwatch: Colors.teal,
      ),
      home: const PostsPage(),
    );
  }
}

// 1. Modelo de Dados (Model)
class Post {
  final int id;
  final String title;
  final String body;

  Post({required this.id, required this.title, required this.body});

  factory Post.fromJson(Map<String, dynamic> json) {
    return Post(
      id: json['id'],
      title: json['title'],
      body: json['body'],
    );
  }
}

// 2. Serviço de Busca de Dados (Service)
Future<List<Post>> fetchPosts() async {
  const url = 'https://jsonplaceholder.typicode.com/posts';
  final response = await http.get(Uri.parse(url));

  if (response.statusCode == 200) {

```

```

// Decodifica a string JSON para uma lista dinâmica
final List<dynamic> jsonResponse = jsonDecode(response.body);

// Mapeia cada item da lista para um objeto Post
return jsonResponse.map((json) => Post.fromJson(json)).toList();
} else {
// Lança uma exceção em caso de erro
throw Exception('Falha ao carregar posts. Status: ${response.statusCode}');
}
}

// 3. Widget de Interface (View)
class PostsPage extends StatelessWidget {
  const PostsPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Posts do JSONPlaceholder'),
      ),
      body: Center(
        // O Widget FutureBuilder gerencia o estado assíncrono
        child: FutureBuilder<List<Post>>(
          future: fetchPosts(), // O Future a ser observado
          builder: (context, snapshot) {
            // Estado 1: Carregando
            if (snapshot.connectionState == ConnectionState.waiting) {
              return const CircularProgressIndicator();
            }

            // Estado 2: Erro
            else if (snapshot.hasError) {
              return Padding(
                padding: const EdgeInsets.all(16.0),
                child: Text('Erro ao carregar: ${snapshot.error}', textAlign:
TextAlign.center),
              );
            }

            // Estado 3: Sucesso
            else if (snapshot.hasData) {
              final posts = snapshot.data!;
              return ListView.builder(
                itemCount: posts.length,
                itemBuilder: (context, index) {

```

```

        final post = posts[index];
        return Card(
            margin: const EdgeInsets.symmetric(vertical: 8, horizontal:
16),
            child: ListTile(
                leading: CircleAvatar(child: Text('${post.id}')),
                title: Text(post.title, style: const
TextStyle(fontWeight: FontWeight.bold)),
                subtitle: Text(post.body, maxLines: 2, overflow:
TextOverflow.ellipsis),
                onTap: () {
                    // Exemplo de navegação para detalhes (Capítulo 5)
                    Navigator.push(
                        context,
                        MaterialPageRoute(
                            builder: (context) => PostDetailPage(post: post),
                        ),
                    );
                },
            ),
        );
    },
);
}

// Estado 4: Sem Dados
return const Text('Nenhum post encontrado.');
```

```

// Widget de Detalhes (Exemplo de Navegação)
class PostDetailPage extends StatelessWidget {
    final Post post;
    const PostDetailPage({super.key, required this.post});

    @override
    Widget build(BuildContext context) {
        return Scaffold(
            appBar: AppBar(
                title: Text(post.title),
            ),
            body: SingleChildScrollView(
```

```

padding: const EdgeInsets.all(16.0),
child: Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Text('ID: ${post.id}', style: const TextStyle(fontSize: 16, color:
Colors.grey)),
    const SizedBox(height: 10),
    Text(post.body, style: const TextStyle(fontSize: 18)),
  ],
),
),
);
}
}

```

C.2. Explicação dos Conceitos Integrados

Conceito	Onde é Usado	Explicação
<code>http</code>	<code>fetchPosts()</code>	Usado para realizar a requisição <code>GET</code> para a API.
<code>dart:convert</code>	<code>jsonDecode()</code>	Converte a resposta do servidor (String JSON) em um objeto Dart.
<code>Post.fromJson</code>	<code>Post</code> class	Construtor de fábrica que mapeia os campos do JSON para as propriedades da classe Dart.
<code>FutureBuilder</code>	<code>PostsPage</code> body	Observa o <code>Future</code> retornado por <code>fetchPosts()</code> e decide qual Widget exibir (carregando, erro ou lista).
<code>snapshot.connectionState</code>	<code>FutureBuilder</code>	Usado para verificar o estado atual da conexão assíncrona.
<code>snapshot.hasData</code>	<code>FutureBuilder</code>	Indica que o <code>Future</code> foi concluído com sucesso e os dados estão disponíveis.
<code>ListView.builder</code>	<code>FutureBuilder</code> (Sucesso)	Exibe a lista de objetos <code>Post</code> de forma eficiente.

Próximo Passo: Formatação Final da Apostila.

Conclusão

Parabéns! Você chegou ao final desta apostila e, com isso, adquiriu uma base sólida para iniciar sua carreira como desenvolvedor Flutter.

Cobrimos os fundamentos essenciais, desde a configuração do ambiente até tópicos mais avançados como gerenciamento de estado e consumo de API. O mais importante agora é continuar praticando.

Próximos Passos Sugeridos:

1. **Construa:** Comece a construir seus próprios aplicativos, mesmo que sejam simples. Tente replicar aplicativos que você usa diariamente.
2. **Explore:** Mergulhe mais fundo em um gerenciador de estado (como Riverpod ou Bloc) e em soluções de banco de dados (como Hive ou Isar).
3. **Comunidade:** Participe de comunidades online (Reddit, Discord, grupos de Telegram) para tirar dúvidas e aprender com outros desenvolvedores.

Lembre-se: a melhor maneira de aprender a programar é programando.

Referências

Esta apostila foi elaborada com base em documentações oficiais e melhores práticas da comunidade Flutter.

1. [Documentação Oficial do Flutter](#)
2. [Documentação Oficial do Dart](#)
3. [Pacote http no Pub.dev](#)
4. [Pacote shared_preferences no Pub.dev](#)
5. [JSONPlaceholder - API de Teste](#)